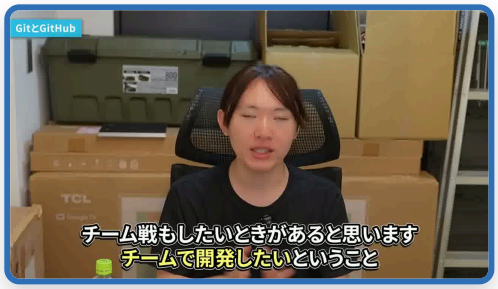
講義ダイジェスト ／ バイブコーディング超入門 第5回

# GitとGitHub
# 記録と共有をつなぐ

役割の違いから、共同開発で迷わないための基本概念まで。




**01:08**　壊れた機能を、元に戻したい。




**01:17**　他の人と、一緒に開発したい。

話者は「変更を戻す」「チームで開発する」「作ったものを公開する」を、学ぶ目的として挙げる。 [00:00–01:36]

ひとことで言うと

## Gitで手元の履歴を管理し、GitHubで共有する。記録と共有は、別の役割。 [01:36–02:17] [02:18–02:45]

### 01
**まず役割を分ける**

Gitだけでも履歴管理はできる。
GitHubは共有の場。 [02:18–02:45]

### 02
**5つの概念でつかむ**

分岐・記録・送信・依頼・取り込み
を理解する。 [03:03–04:03]

### 03
**共同開発につなげる**

他の人の変更は、手元へ取り込む。
[08:49–10:09]

# 01 手元の記録と、共有の場所を分ける

役割の違い　01:37–03:03

### Git

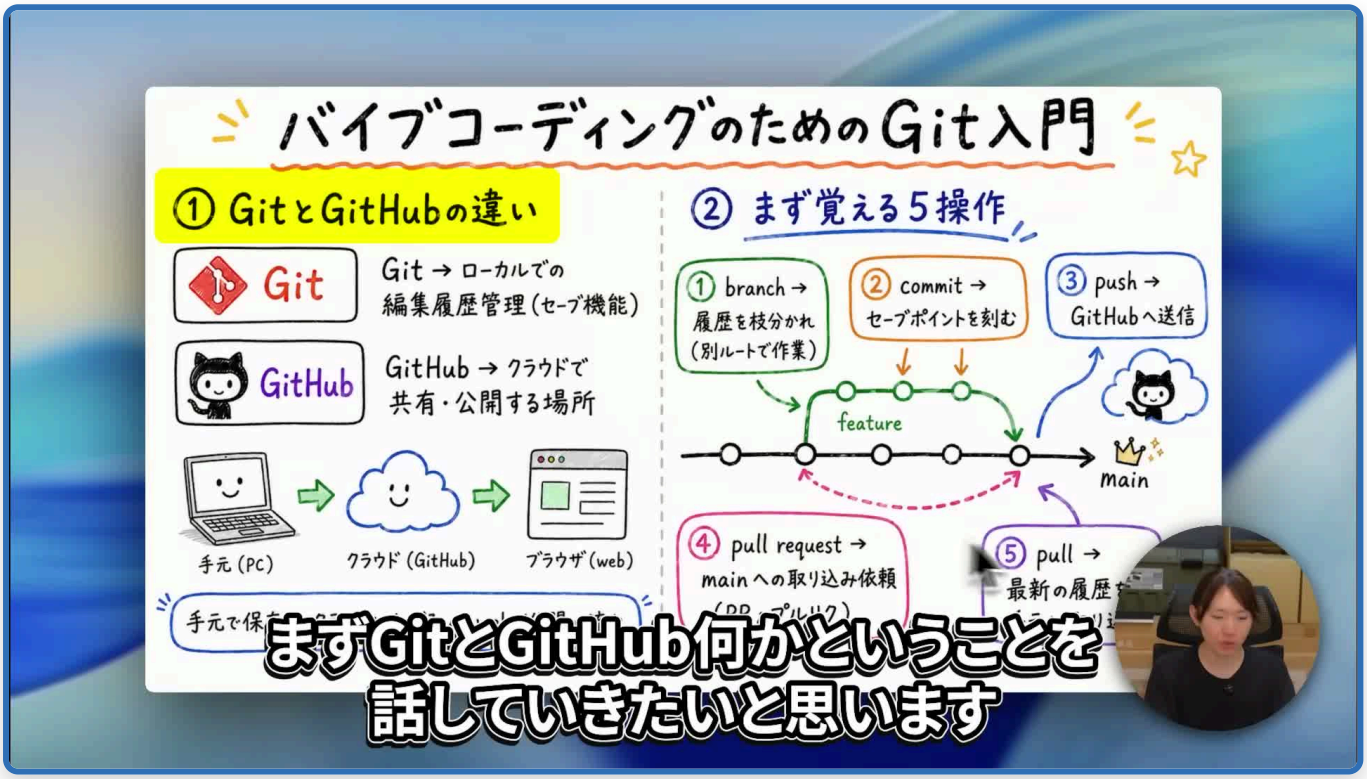
**自分のパソコンで履歴を管理**

「ローカル」は手元のパソコンのこと。話者は、編集の履歴を残すバージョン管理を、ゲームの「セーブポイント」に例えている。 [01:36–02:17]

### GitHub

**コードや履歴を共有する場所**

Gitで管理するものをクラウド上で共有・公開し、やり取りする場。GitHubを使わなくても、Gitによる履歴管理はできる。 [02:18–02:45]




**01:49**　講義の全体図。左にGitとGitHubの違い、右に入門用の5項目。話者は、説明図をChatGPTで作成したと述べている。 [01:36–02:17] [03:03–04:03]

## GitHubへの送信と、Web公開は同じ段階ではない

動画で紹介する公開の流れは、手元で保存し、GitHubへ送り、そこから別のサーバーへソースコードを移してWebで公開する、というもの。 [02:45–03:03]

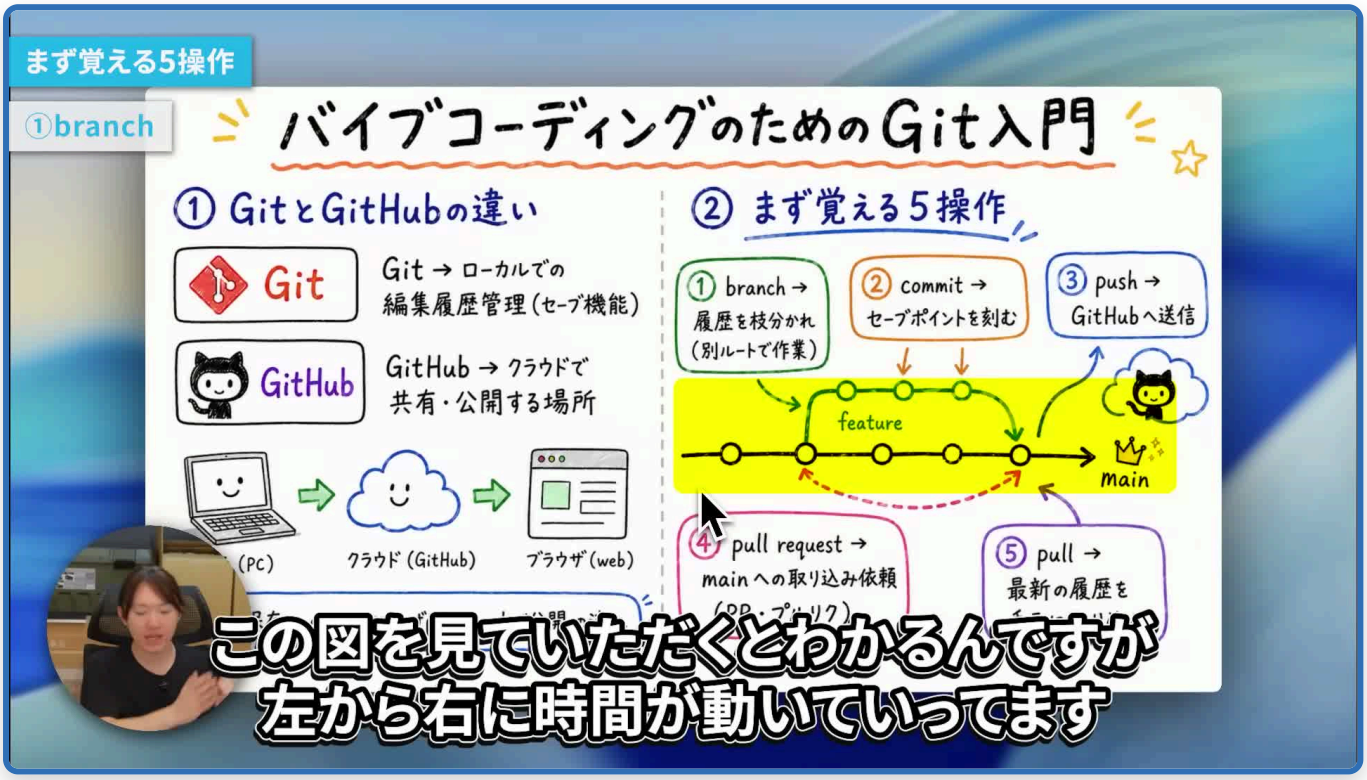
| 手元で保存 | GitHubへ送信 | 別サーバーへ配置 |
|---|---|---|
| 自分のパソコン | コードや履歴を共有 | 動画で説明するWeb公開 |

「GitHubへ送ること」と「サイトを動かすために配置すること」を、分けて理解する。 [02:45–03:03]

# 02 履歴は「分ける」と「残す」で読む

基本概念・前半　03:03–06:51

まず覚えるのは、ブランチ・コミット・プッシュ・PR・プル。これは本講義が入門向けに選んだ5項目で、Gitの全機能でも、すべてが同種のGitコマンドでもない。話者はAIの支援を前提に、学習内容を絞っている。 [03:03–04:03]




**04:53**　黄色で強調された履歴図。時間は左から右へ進み、黒いmainから緑のfeatureへ分岐する。 [04:03–06:13]

## 01　ブランチ

### 別ルートで作業する

履歴を枝分かれさせ、分岐先で変更を進める。うまくいった変更は元の流れへ取り込める、と説明される。 [04:03–06:13]

図の黒い線はmain、緑の線はfeature。復元や統合の具体的操作は、本編では示されない。 [04:03–06:13]

## 02　コミット

### 作業の区切りを記録する

図の小さな丸が「セーブポイント」に相当する。編集して、区切りのよい状態でコミットし、再び編集とコミットを重ねる。 [06:14–06:50]

ブランチは作業ルートの分岐、コミットは履歴に残す区切り。同じ役割ではない。 [04:03–06:13] [06:14–06:50]

**読み方**　**線の分岐と、丸の積み重ねを見分ける。「分ける」と「残す」を区別すると、図の意味がつかみやすい。** [04:03–06:13] [06:14–06:50]

# 03 基本概念・後半　06:51–08:49
# 送信・依頼・取り込みは別の役割

### 03　プッシュ：共有先へ送る

手元で行った作業の履歴を、GitHubへ送信する。講義図の青い矢印は、共有先へ送る向き。 [06:51–07:12]

**手元のPC → GitHub**

### 05　プル：共有先から取り込む

動画では、共有先で更新されたmainの履歴を、手元へ取り込む例として説明している。 [08:04–08:49]

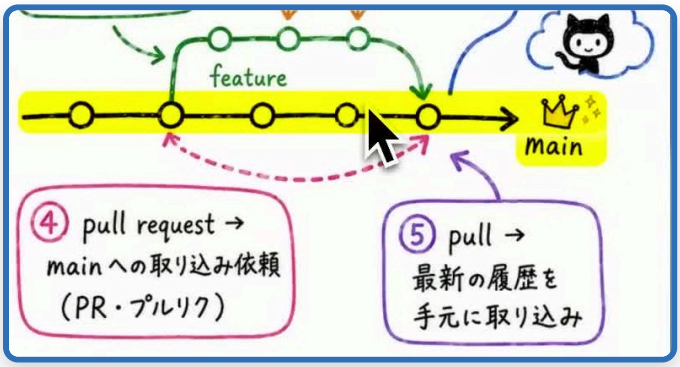
[外部資料: Git - git-pull Documentation]

**GitHub → 手元のPC**

## 04　プルリクエスト（PR）

### 変更の取り込みを「依頼」する

講義では、自分の変更をmainへ入れてほしいと依頼し、GitHub上で確認やレビューをする仕組みとして紹介。「プルリク」とも呼ばれる。 [07:13–08:03] [外部資料: プルリクエストについて - GitHubドキュメント]




**07:26**　mainを黄色で強調。PRは取り込み依頼、pullは手元への取り込みを示す。 [07:13–08:03]
[外部資料: プルリクエストについて - GitHubドキュメント]
[08:04–08:49] [外部資料: Git - git-pull Documentation]

**公式資料による補足 ／ GitHub**

**PRの作成だけで、マージは完了しない。**PRは別ブランチへの変更の取り込みを提案し、話し合いやレビューを行う仕組み。実際の取り込みがマージに当たる。 [07:13–08:03]

[外部資料: プルリクエストについて - GitHubドキュメント]

**公式資料による補足 ／ Git**

pullは、取得した変更を**現在のブランチへ統合**する。引数を省略した場合、取り込み元は現在のブランチの上流設定が基準になる。常にmainを更新する命令ではないため、現在のブランチと取り込み元を確認する。 [08:04–08:49] [外部資料: Git - git-pull Documentation]

参照は公式マニュアル2.53.0。映像内のGitの版は確認できず、版ごとの既定動作の違いには踏み込まない。

# 04 共有先が最新でも、手元は別

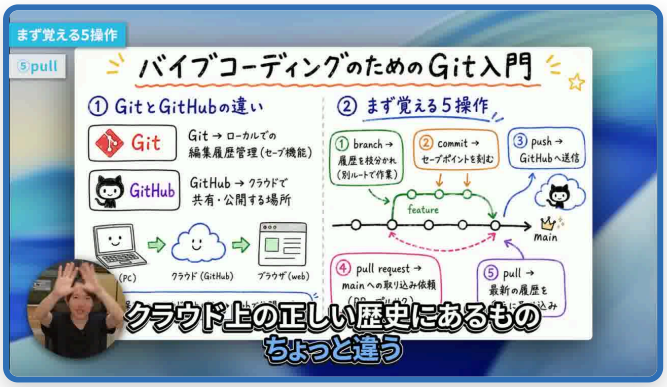
二人開発の例　08:49–10:10 ／ 振り返り　11:31–11:49

二人が別々のブランチで開発し、共有先で両方の変更をマージした例。自分のPCには、もう一人の変更がまだないため、pullで取り込む必要がある。 [08:49–10:09]

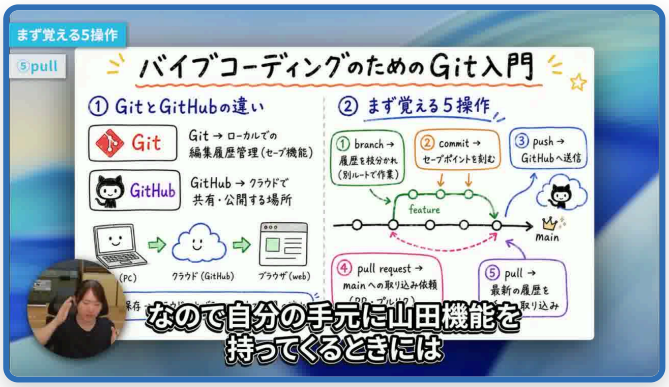
**講義内の仮例を整理 ／ 実際の操作記録ではない**

| どこの状態か | 自分の変更 | もう一人の変更 |
|---|---|---|
| 自分のPC（pull前） | ある | まだない |
| GitHubのmain（両方マージ後） | ある | ある |
| 自分のPC（pull後） | ある | 取り込まれる |

共有先に統合された変更を、手元に取り込む段階が必要になる。 [08:49–10:09]




**09:24**　共有先と手元の違いを説明する場面。実際の同期操作ではない。




**09:43**　他の人の変更を取り込むため、pullが必要だと説明する場面。 [08:49–10:09]

## 5項目を、前後関係でつなぐ

ブランチ（分岐） → コミット（記録） → プッシュ（送信） → PR（取り込み依頼） → マージ（レビューを経て統合） → **プル（手元へ取り込み）**

講義の振り返りを概念の流れとして整理。PRを出した直後に統合が済むのではなく、レビュー・マージを経て共有先へ反映される段階を間に置く。 [11:30–11:49] [07:13–08:03] [外部資料: プルリクエストについて - GitHubドキュメント]

# 05 コンフリクト　10:10–11:30
# 競合は、Gitが自動で決められないとき

話者は、ホームページのテーマカラーを別々の色に変える仮例で、コンフリクト（衝突）を説明する。 [10:10–11:30] [外部資料: マージ競合 - GitHubドキュメント]

**自分の変更**

## 赤にする

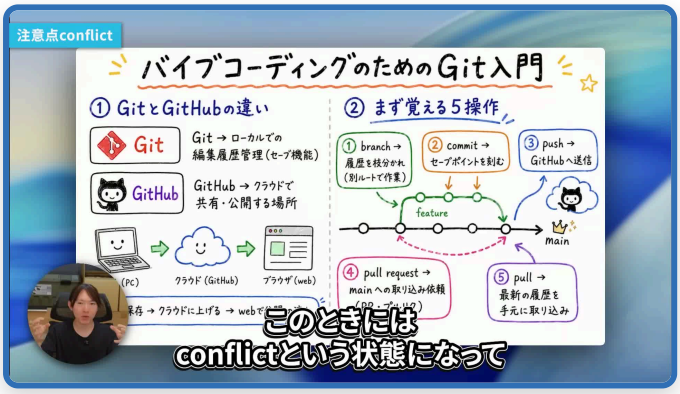
自分のブランチで、テーマカラーを変更する。

**もう一人の変更**

## 青にする

別のブランチで、テーマカラーを変更する。

赤と青は講義で示した仮例。実際に競合したファイルや、競合を解消する操作は映っていない。 [10:10–11:30]
[外部資料: マージ競合 - GitHubドキュメント]




**10:58**　コンフリクトの説明場面。背景は概念図であり、実際の競合エラー画面ではない。 [10:10–11:30]
[外部資料: マージ競合 - GitHubドキュメント]

## 最終的な内容は、人が決める。

講義では、どちらの色を採用するかを決めてから取り込む、と説明される。単に依頼を採用するだけで済まない場面がある。 [10:10–11:30]
[外部資料: マージ競合 - GitHubドキュメント]

**公式資料による補足 ／ GitHub**

### 競合には「自動統合できない」という条件がある

GitHub公式資料によると、競合はGitが変更を安全に自動統合できないときに起こる。同じファイルの同じ行への異なる変更や、同じファイルの編集と削除が重なる場合などが典型例。 [10:10–11:30]
[外部資料: マージ競合 - GitHubドキュメント]

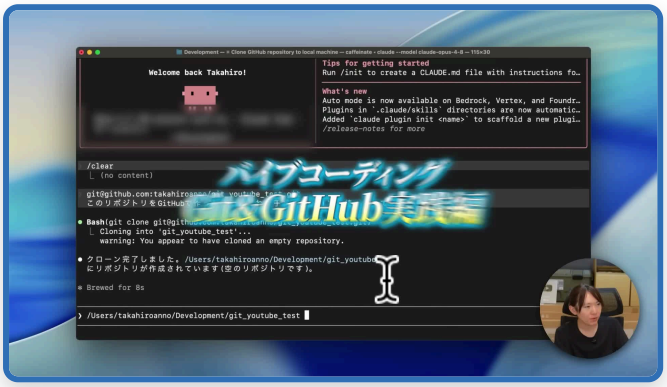
**色の方針が違うだけ、双方がpushしただけで、必ず競合するわけではない。**動画の仮例と、競合が発生する技術的な条件を分けて捉える。 [10:10–11:30] [外部資料: マージ競合 - GitHubドキュメント]

# 06 実践編の予告　11:49–12:32
# 末尾の操作画面は、次回への予告

readme.mdの2行作成、別のメモファイルのコミット、計2件の履歴が映る。ただし途中にカットがあり、5項目を一通り実演した映像ではない。「実践編・近日公開」の告知後は挨拶と終了画面になる。 [11:49–12:32]

## 予告に映る道具

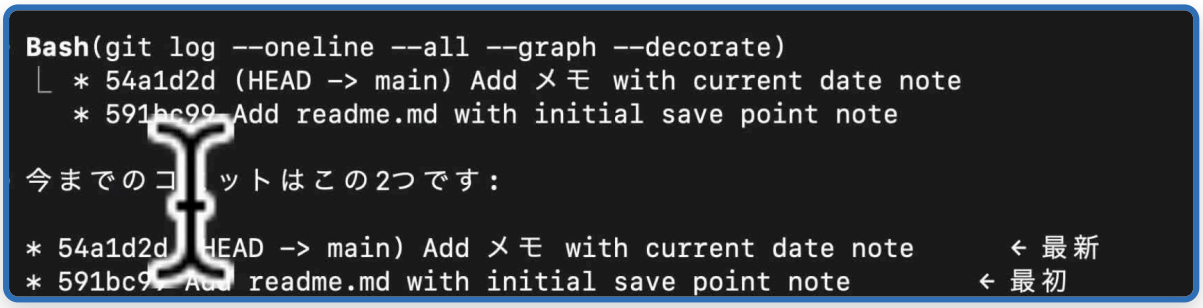
**Claude Codeの端末とGit。** 導入方法・認証設定・必要バージョンは説明されない。 [11:49–12:32]




**11:49**　Claude Codeを実行する端末。 [11:49–12:32]




**12:12**　実践編「近日公開」の告知。




**12:08**　Gitの履歴表示を拡大。readme.md追加とメモ追加の2件が並び、最新に「HEAD → main」が付く。 [11:49–12:32]

---

まとめ

# 記録・共有・取り込みを区別する

**01**　Gitは手元の履歴管理、GitHubは共有の場。役割を分けて理解する。 [01:36–02:17] [02:18–02:45]

**02**　分岐・記録・送信・依頼・取り込みを、流れで理解する。PRの後にはレビュー・マージの段階がある。 [11:30–11:49]

**03**　競合は、自動統合できないときに起こる。人が最終的な内容を決める必要がある。 [10:10–11:30]
[外部資料: マージ競合 - GitHubドキュメント]